

Python Pandas - Filling Missing Data

Filling missing data is a process of replacing the missing (NaN) values with meaningful alternatives. Whether you want to replace missing values with a constant value, or propagate the values forward or backward, Pandas has built-in functions to achieve this.

Replacing missing values with a scalar.

Forward and backward filling.

Using a specified limit for filling.

Replacing Data with the `replace()` method.

Replacing values with regular expressions.

Filling Missing Data with Scalar Value

The `fillna()` method in Pandas is used to fill missing values (NA or NaN) with a scalar value, such as any specific number.

Example

```
import pandas as pd
import numpy as np
```

```

data = {"Col1": [3, np.nan, np.nan, 2], "Col2": [1.0, pd.NA, pd.NA, 2.0]}
df = pd.DataFrame(data)
# Display the DataFrame with missing values
print("DataFrame:\n",df)
# Fill missing values with -
f = df.fillna('-')
print("\nNew DataFrame after NaN replaced with '-':\n", f)

```

Output

DataFrame:

	Col1	Col2
0	3.0	1.0
1	NaN	<NA>
2	NaN	<NA>
3	2.0	2.0

New DataFrame after NaN replaced with '0':

	Col1	Col2
0	3.0	1.0
1	-	-
2	-	-
3	2.0	2.0

Filling Missing Values Forward or Backward

You can also propagate the last valid observation forward or backward to fill gaps using the `ffill()` and `bfill()` methods respectively.

`ffill()` : This method fills missing values with the previous valid value.

`bfill()` : This methods fills missing values with the next valid value.

Example: Forward Fill

```
import pandas as pd

df = pd.DataFrame([[9, -3, -2], [-5, 1, 8], [6, 4, -8]],
index=['a', 'c', 'd'], columns=['one', 'two', 'three'])

df = df.reindex(['a', 'b', 'c', 'd', 'e'])

# Display the DataFrame with missing values

print("DataFrame:\n",df)


# Forward Fill the missing values

result = df.ffmpeg()

print("\nNew DataFrame after Forward fill:\n", result)
```

Output

DataFrame:

	one	two	three
a	9.0	-3.0	-2.0
b	NaN	NaN	NaN
c	-5.0	1.0	8.0
d	6.0	4.0	-8.0
e	NaN	NaN	NaN

New DataFrame after Forward fill:

	one	two	three
a	9.0	-3.0	-2.0
b	9.0	-3.0	-2.0
c	-5.0	1.0	8.0
d	6.0	4.0	-8.0
e	6.0	4.0	-8.0

Example: Backward Fill

```
import pandas as pd

df = pd.DataFrame([[9, -3, -2], [-5, 1, 8], [6, 4, -8]],
index=['a', 'c', 'd'], columns=['one', 'two', 'three'])

df = df.reindex(['a', 'b', 'c', 'd', 'e'])

print("DataFrame:\n",df)

# Backward Fill the missing values

result = df.bfill()

print("\nNew DataFrame after Backward fill:\n", result)
```

Output

	one	two	three
a	9.0	-3.0	-2.0
b	NaN	NaN	NaN
c	-5.0	1.0	8.0
d	6.0	4.0	-8.0
e	NaN	NaN	NaN

New DataFrame after Backward fill:

	one	two	three
a	9.0	-3.0	-2.0
b	-5.0	1.0	8.0
c	-5.0	1.0	8.0
d	6.0	4.0	-8.0
e	NaN	NaN	NaN

Limiting the Number of Fills

You can also control the limit of how many consecutive missing values are filled by specifying the limit parameter.

Example

```
import pandas as pd

df = pd.DataFrame([[9, -3, -2], [-5, 1, 8], [6, 4, -8]],
index=['a', 'c', 'd'], columns=['one', 'two', 'three'])

df = df.reindex(['a', 'b', 'd', 'e', 'f'])

# Display the DataFrame with missing values
print("DataFrame:\n",df)

# Forward Fill the missing values with limit
```

```
result = df.ffill(limit=1)
```

```
print("\nNew DataFrame after Forward fill:\n", result)
```

Output

	one	two	three
a	9.0	-3.0	-2.0
b	NaN	NaN	NaN
d	6.0	4.0	-8.0
e	NaN	NaN	NaN
f	NaN	NaN	NaN

New DataFrame after Forward fill:

	one	two	three
a	9.0	-3.0	-2.0
b	9.0	-3.0	-2.0
d	6.0	4.0	-8.0
e	6.0	4.0	-8.0
f	NaN	NaN	NaN

Replacing Data with the replace() method

Many times, we have to replace a generic value with some specific value. We can achieve this by applying the replace() method.

Replacing NA with a scalar value is equivalent behavior of the fillna() function.

Example

Here is the example of replacing the generic values using the replace() method.

```
import pandas as pd

df= pd.DataFrame({'one':[10,20,30,40,50,2000], 'two':[1000,0,30,40,50,60]})

# Replace the generic values

print(df.replace({1000:10,2000:60}))
```

Output

	one	two
0	10	10
1	20	0
2	30	30
3	40	40


```
4    50    50
5    60    60
```

Replacing Missing Data Using Regular Expressions

You can also use regex patterns to replace the missing values in your data with the `replace()` method.

Example

```
import pandas as pd
```

```
df = pd.DataFrame(['$40,000*', '$40000 conditions attached'], columns=['P'])
```

```
print(df)
```

```
      P
0  $40,000*
1 $40000 conditions attached
```

```
df['P'] = df['P'].str.replace(r'\D+', '', regex=True).astype('int')
```

```
print(df)
```

Output

P

0 40000

1 40000

since \D matches any character that is not a decimal digit.

groupby()

The `groupby()` method in Pandas is used for splitting the data into groups based on some criteria, applying a function to each group independently, and then combining the results into a single object. This is often referred to as the "split-apply-combine" strategy.

Example

```
import pandas as pd

data = {'Category': ['A', 'B', 'A', 'C', 'B', 'C', 'A'],
        'Value': [10, 15, 12, 8, 20, 5, 18]}

df = pd.DataFrame(data)

group_data = df.groupby('Category')['Value'].sum()

print(group_data)
```

Explanation:

df.groupby('Category'): This step splits the DataFrame df into groups based on the unique values in the 'Category' column. It returns a GroupBy object.

['Value']: This selects the 'Value' column within each of these groups.

.sum(): This applies the sum() aggregation function to the 'Value' column for each group, calculating the total 'Value' for each 'Category'.

Output:

Category

A 40

B 35

C 13

Name: Value, dtype: int64

Other common aggregations with groupby():

.mean(): Calculates the mean of each group.

.count(): Counts the number of non-null values in each group.

.min(): Finds the minimum value in each group.

.max(): Finds the maximum value in each group.

.agg(): Allows applying multiple aggregation functions or custom functions.

Grouping by multiple columns:

You can also group by multiple columns by passing a list of column names to `groupby()`:

```
import pandas as pd
```

```
data = {'Category': ['A', 'B', 'A', 'C', 'B', 'C', 'A'],  
        'SubCategory': ['X', 'Y', 'X', 'Z', 'Y', 'Z', 'Y'],  
        'Value': [10, 15, 12, 8, 20, 5, 18]}
```

```
df = pd.DataFrame(data)
```

```
# Group by 'Category' and then 'SubCategory', and calculate the mean of 'Value'
```

```
group_multi_columns = df.groupby(['Category', 'SubCategory'])['Value'].mean()
```

```
print(group_multi_columns)
```

Output:

Category	SubCategory	
A	X	11.0
	Y	18.0

B	Y	17.5
---	---	------

C	Z	6.5
---	---	-----

Name: Value, dtype: float64

Pandas DataFrames Sorting

Pandas DataFrames can be sorted by column values using the `sort_values()` method or by their index using the `sort_index()` method.

Key Parameters:

by:	Specifies the column(s) to sort by (for <code>sort_values()</code>). Can be a single column name or a list of names.
axis:	Specifies whether to sort by rows (0 or 'index', default) or columns (1 or 'columns').
ascending:	Boolean or list of Booleans. True for ascending order, False for descending. If a list, it applies to corresponding columns in by.
inplace:	Boolean. If True, modifies the DataFrame in place. Default is False.
na_position:	Controls the placement of NaN values ('first' or 'last'). Default is 'last'.

➤ **Sorting by Column Values (sort_values()):**

Single Column Ascending.

```
import pandas as pd

data = {'Name': ['ABC', 'XYZ', 'PQR'], 'Age': [28, 22, 34]}
df = pd.DataFrame(data)
res = df.sort_values(by='Age')
print(res)
```

Output:

	Name	Age
1	XYZ	22
0	ABC	28
2	PQR	34

Single Column Descending.

```
res = df.sort_values(by='Age', ascending=False)
```

Output:

```
   Name  Age
2  PQR   34
0  ABC   28
1  XYZ   22
```

Multiple Columns: Sorts first by the first column in the list, then by the second, and so on.

```
data = {'Name': ['ABC', 'XYZ', 'PQR', 'XYZ'], 'Score': [85, 92, 78, 95]}
df = pd.DataFrame(data)
res = df.sort_values(by=['Name', 'Score'], ascending=[True, False])
```

This sorts by 'Name' ascending, then by 'Score' descending for rows with the same 'Name'.

Output:

	Name	Score
0	ABC	85
2	PQR	78
3	XYZ	95
1	XYZ	92

inplace parameter: If inplace=True, the original DataFrame is modified directly, and no new DataFrame is returned.

```
import pandas as pd
```

```
data = {'Name': ['ABC', 'XYZ', 'PQR'], 'Age': [28, 22, 34]}
```

```
df = pd.DataFrame(data)
```

```
df.sort_values(by='Age', inplace=True)
```

```
print(df)
```

➤ **Sorting by Index (sort_index()):**

Ascending by Index.

```
import pandas as pd

data = {'Name': ['ABC', 'XYZ', 'PQR'], 'Age': [28, 22, 34]}
df = pd.DataFrame(data)
res = df.sort_index()
print(res)
```

Descending by Index.

```
res = df.sort_index(ascending=False)
```

Sorting Columns (by column labels).

```
import pandas as pd

data = {'Name': ['ABC', 'XYZ', 'PQR'], 'Age': [28, 22, 34]}
df = pd.DataFrame(data)
res = df.sort_index(axis=1)
print(res)
```

Pandas Date Functionality

In time-series data analysis, especially in financial domains, date functionality plays a crucial role. Pandas provides robust tools to work with dates, allowing you to generate date sequences, manipulate date frequencies, and work with business days.

Pandas date functionality is divided into four primary concepts –

Date times: Represent specific points in time, like `datetime.datetime` from the standard library.

Time deltas: Represent duration in time, similar to `datetime.timedelta`.

Time spans: Define a span of time with a specific frequency, like months or years.

Date offsets: Represent relative time changes that respect calendar rules. Similar to `dateutil.relativedelta.relativedelta` from the `dateutil` package.

Generating a Sequence of Dates

You can create a range of dates using the `date.range()` function. By specifying the start date, number of periods, and frequency, you can generate a sequence of dates.

Example

```
import pandas as pd

print(pd.date_range('6/1/2025', periods=5))
```

Output

```
DatetimeIndex(['2025-06-01', '2025-06-02', '2025-06-03', '2025-06-04', '2025-06-05'],  
              dtype='datetime64[ns]', freq='D')
```

Load Files Into a DataFrame

If your data sets are stored in a file, Pandas can load them into a DataFrame.

Pandas Read CSV

A simple way to store big data sets is to use CSV files (comma separated files).

CSV files contains plain text and is a well know format that can be read by everyone including Pandas.

Example

Load a comma separated file (CSV file) into a DataFrame:

```
import pandas as pd  
  
df = pd.read_csv('data.csv')  
  
print(df)
```

Note: use `to_string()` to print the entire DataFrame.

```
print(df.to_string())
```

If you have a large DataFrame with many rows, Pandas will only return the first 5 rows, and the last 5 rows:

Example

Print the DataFrame without the `to_string()` method:

```
import pandas as pd  
  
df = pd.read_csv('data.csv')  
  
print(df)  
  
max_rows
```

The number of rows returned is defined in Pandas option settings.

You can check your system's maximum rows with the `pd.options.display.max_rows` statement.

Analyzing a sample CSV file with Pandas in Python typically involves these steps:

```
import pandas as pd  
  
df = pd.read_csv("data.csv")
```

Explore the Data:

➤ **View the first few rows:**

```
print(df.head())
```

Example

Printing the first 10 rows of the DataFrame:

```
import pandas as pd  
  
df = pd.read_csv('data.csv')
```

```
print(df.head(10))
```

Note: if the number of rows is not specified, the head() method will return the top 5 rows.

There is also a tail() method for viewing the last rows of the DataFrame.

The tail() method returns the headers and a specified number of rows, starting from the bottom.

Example

Print the last 5 rows of the DataFrame:

```
print(df.tail())
```


- **Get a summary of the DataFrame (including data types and non-null values):**

```
print(df.info())
```

- **Generate descriptive statistics for numerical columns.**

```
print(df.describe())
```

- **Check the dimensions of the DataFrame (rows, columns):**

```
print(df.shape)
```

- **View column names.**

```
print(df.columns)
```

- **Perform Basic Analysis (Examples):**

Select a specific column:

```
print(df['column_name'])
```

➤ **Filter data based on a condition.**

```
filtered_df = df[df['column_name'] > value]

print(filtered_df.head())
```

➤ **Group data and apply aggregation functions.**

```
group_data = df.groupby('category_column')['numerical_column'].mean()

print(grouped_data)
```

➤ **Count unique values in a column.**

```
print(df['column_name'].value_counts())
```

These steps provide a foundation for analyzing a sample CSV file using the Pandas library. You can expand upon this by performing more complex operations like data cleaning, transformation, and visualization as needed.

Example

- **Check the number of maximum returned rows:**

```
import pandas as pd

print(pd.options.display.max_rows)
```

You can change the maximum rows number with the same statement.

Example

Increase the maximum number of rows to display the entire DataFrame:

```
import pandas as pd

pd.options.display.max_rows = 9999

df = pd.read_csv('data.csv')

print(df)
```

Pandas Read JSON

Big data sets are often stored, or extracted as JSON.

JSON is plain text, but has the format of an object, and is well known in the world of programming, including Pandas.

We will be using a JSON file called 'data.json'.

Example

Load the JSON file into a DataFrame:

```
import pandas as pd  
  
df = pd.read_json('data.json')  
  
print(df.to_string())
```

Note: use `to_string()` to print the entire DataFrame.

Dictionary as JSON

JSON = Python Dictionary

JSON objects have the same format as Python dictionaries.

If your JSON code is not in a file, but in a Python Dictionary, you can load it into a DataFrame directly:

Example

Load a Python Dictionary into a DataFrame:

```
import pandas as pd
```

```
data = {  
    "Duration":{  
        "0":60,  
        "1":60,  
        "2":60,  
        "3":45,  
        "4":45,  
        "5":60
```

},

"Pulse":{

"0":110,

"1":117,

"2":103,

"3":109,

"4":117,

"5":102

},

"Maxpulse":{

"0":130,

"1":145,

"2":135,

"3":175,

"4":148,

"5":127

},

"Calories":{

"0":409,

```
"1":479,  
"2":340,  
"3":282,  
"4":406,  
"5":300  
}  
}  
  
df = pd.DataFrame(data)  
  
print(df)
```

Pandas - Cleaning Empty Cells

Empty Cells

Empty cells can potentially give you a wrong result when you analyze data.

Remove Rows

One way to deal with empty cells is to remove rows that contain empty cells.

This is usually OK, since data sets can be very big, and removing a few rows will not have a big impact on the result.

Example

Return a new Data Frame with no empty cells:

```
import pandas as pd

df = pd.read_csv('data.csv')

new_df = df.dropna()

print(new_df.to_string())
```

By default, the `dropna()` method returns a new DataFrame, and will not change the original.

If you want to change the DataFrame, use the `inplace = True` argument:

Example

Remove all rows with NULL values:

```
import pandas as pd

df = pd.read_csv('data.csv')

df.dropna(inplace = True)

print(df.to_string())
```

Now, the `dropna(inplace = True)` will NOT return a new DataFrame, but it will remove all rows containing NULL values from the DataFrame.

Replace Empty Values

Another way of dealing with empty cells is to insert a new value instead.

This way you do not have to delete entire rows just because of some empty cells.

The fillna() method allows us to replace empty cells with a value:

Example

Replace NULL values with the number 130:

```
import pandas as pd

df = pd.read_csv('data.csv')

df.fillna(130, inplace = True)
```

Replace Only For Specified Columns

The example above replaces all empty cells in the whole Data Frame.

To only replace empty values for one column, specify the column name for the DataFrame:

Example

Replace NULL values in the "Phone 1" columns with the number 130:

```
import pandas as pd

df = pd.read_csv('data.csv')

df.fillna({"Phone 1": 130}, inplace=True)
```

Replace Using Mean, Median, or Mode

A common way to replace empty cells, is to calculate the mean, median or mode value of the column.

Pandas uses the `mean()` `median()` and `mode()` methods to calculate the respective values for a specified column:

Example

Calculate the MEAN, and replace any empty values with it:

```
import pandas as pd

df = pd.read_csv('data.csv')

x = df["Index"].mean()

df.fillna({"Index": x}, inplace=True)
```

Mean = the average value (the sum of all values divided by number of values).

Example

Calculate the MEDIAN, and replace any empty values with it:

```
import pandas as pd

df = pd.read_csv('data.csv')

x = df["Index"].median()

df.fillna({"Index": x}, inplace=True)
```

Median = the value in the middle, after you have sorted all values ascending.

Example

Calculate the MODE, and replace any empty values with it:

```
import pandas as pd

df = pd.read_csv('data.csv')

x = df["Index"].mode()[0]

df.fillna({"Index": x}, inplace=True)
```